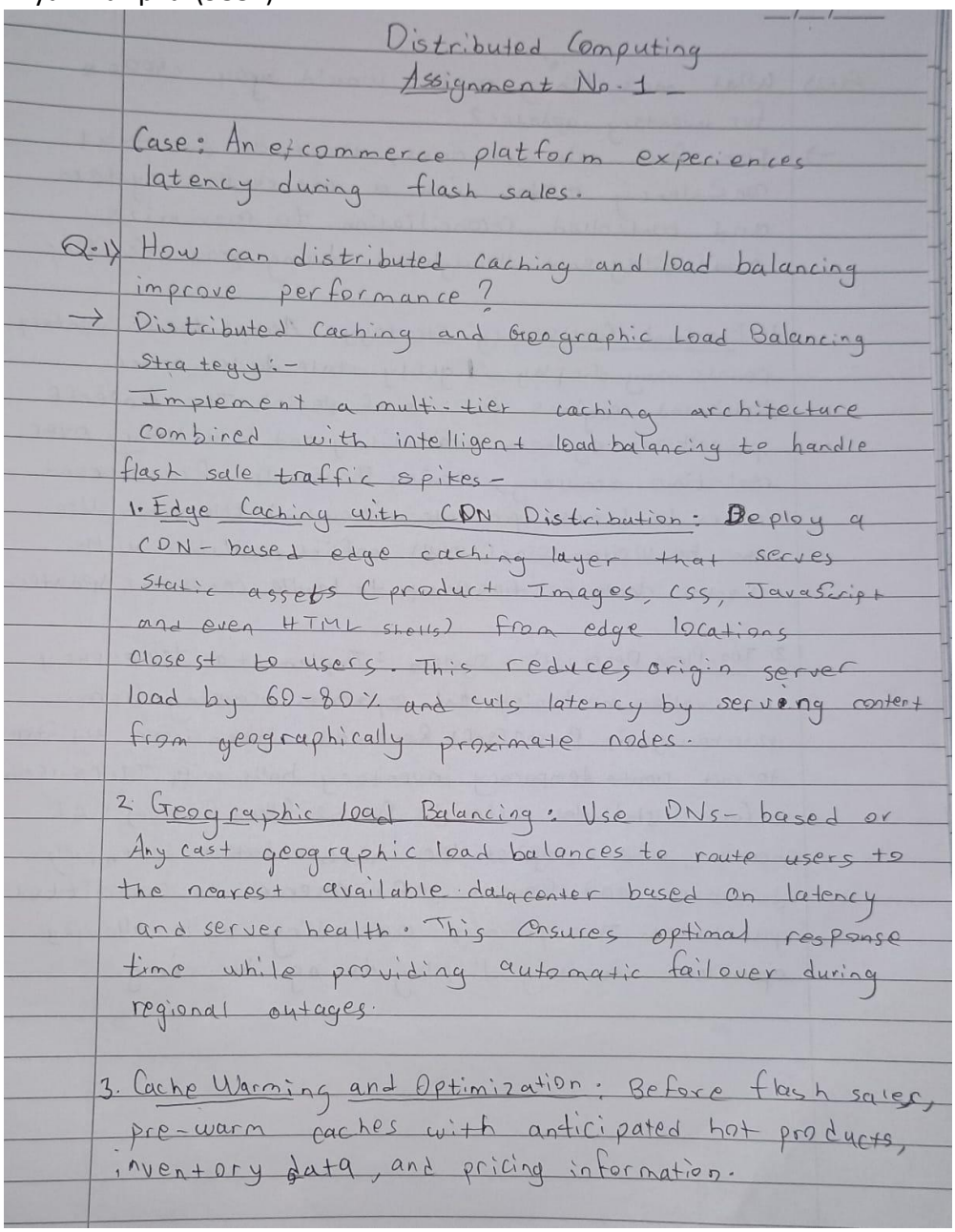
# DC assignment 2

Case: An e-commerce platform experiences latency during flash sales.
How can distributed caching and load balancing improve performance?
What consistency model would you choose for inventory updates**?**

Shwen Coutinho(9881)
Aryan Daripkar(9882)
Jonathan Gomes(9900)
Mark Lopes(9913)
Vivian Ludrick(9914)
Mrugank Worlikar(9938)

Aryan Daripkar(9882)



Distributed Computing
Assignment No. 1

Case: An e-commerce platform experiences latency during flash sales.

Q.1) How can distributed caching and load balancing improve performance?

→ Distributed Caching and Geographic Load Balancing Strategy:-
Implement a multi-tier caching architecture combined with intelligent load balancing to handle flash sale traffic spikes-

1. Edge Caching with CDN Distribution: Deploy a CDN-based edge caching layer that serves static assets (product Images, CSS, JavaScript and even HTML shells) from edge locations closest to users. This reduces origin server load by 60-80% and cuts latency by serving content from geographically proximate nodes.

2. Geographic load Balancing: Use DNS-based or Anycast geographic load balances to route users to the nearest available datacenter based on latency and server health. This ensures optimal response time while providing automatic failover during regional outages.

3. Cache Warming and Optimization: Before flash sales, pre-warm caches with anticipated hot products, inventory data, and pricing information.

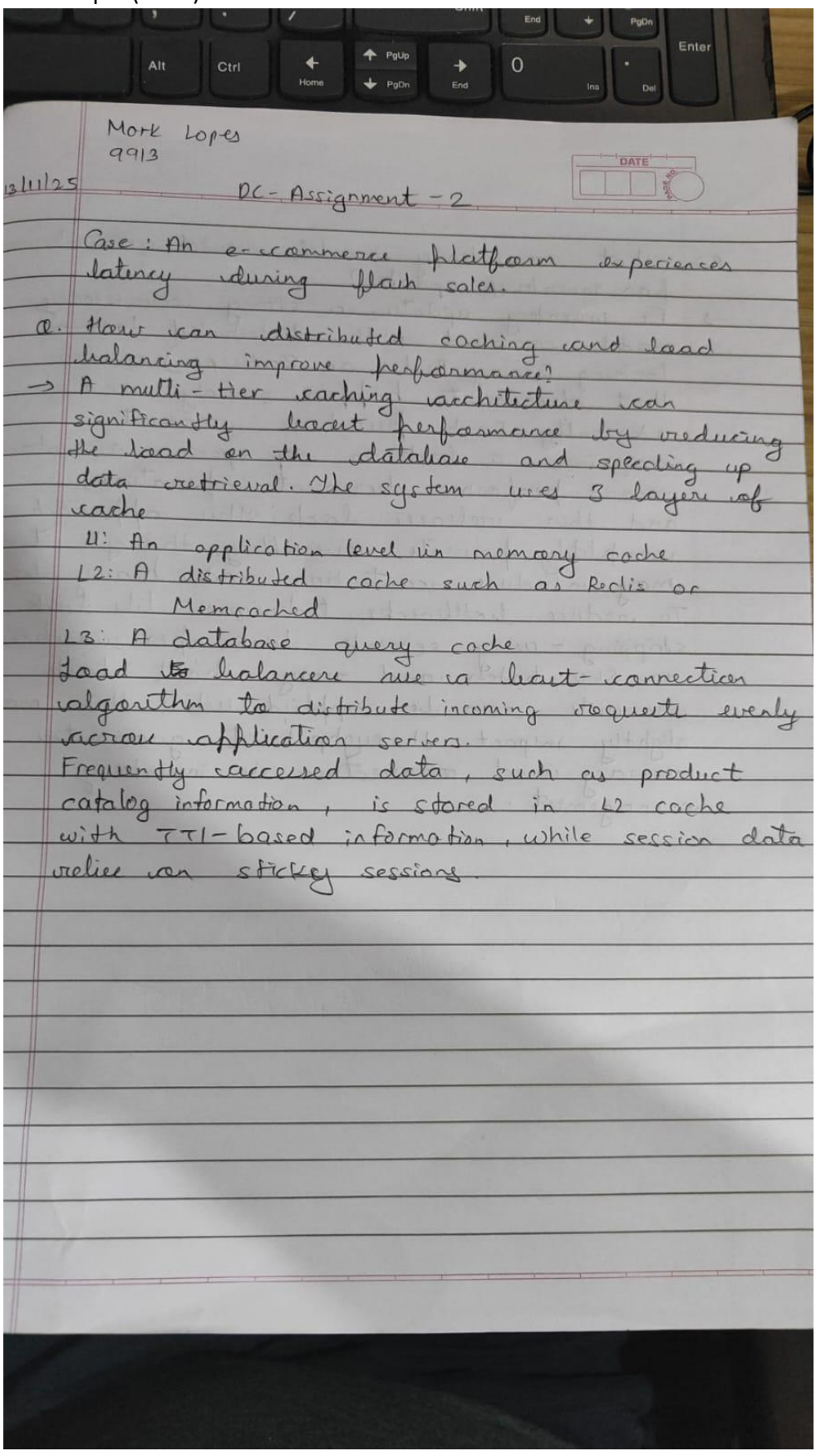
Q.2) What consistency model would you choose for inventory updates?

→ For flash sale scenarios, adopt an eventual consistency model with a reservation system and controlled reconciliation to maximize availability and throughput during peak traffic.

1. Eventual Consistency Strategy: Accept that inventory counts may display slightly stale data across distributed regions for brief periods. This tradeoff prioritizes system availability and low-latency over real-time accuracy, allowing thousands of concurrent users to browse products without bottlenecks. Use the last-write-wins (LWW) conflict resolution with vector clocks or timestamps to handle concurrent updates.

2. Two-Phase Reservation System: Implement a temporary reservation mechanism that separates browsing from purchasing: (i) Phase 1 (Soft Reserve): When users add items to cart, create temporary inventory holds with TTL (5-15 min) in fast distributed cache. (ii) Phase 2 (Hard Reserve): At checkout, validate against the authoritative inventory database and convert soft reserves to committed purchases. This prevents overselling while allowing optimistic inventory display during browsing.

Mark Lopes(9913)



Mark Lopes
9913

13/11/25

DC - Assignment - 2

Case : An e-commerce platform experiences latency during flash sales.

Q. How can distributed caching and load balancing improve performance?

→ A multi-tier caching architecture can significantly boost performance by reducing the load on the database and speeding up data retrieval. The system uses 3 layers of cache

L1: An application level in memory cache
L2: A distributed cache such as Redis or Memcached
L3: A database query cache
Load balancers use a least-connection algorithm to distribute incoming requests evenly across application servers.
Frequently accessed data, such as product catalog information, is stored in L2 cache with TTL-based information, while session data relies on sticky sessions.

Q.2 what consistency model would you choose for inventory updates?

→ For inventory updates, a strong consistency model combined with distributed ~~locking~~ locking (using Redis or Zookeeper) ensures that decrements happen safely and atomically. Each purchase request acquires a lock on ~~set~~ specific inventory record, performs decrement operation, and then releases lock. This approach completely prevents overselling, though it may introduce contention under heavy load. To reduce bottlenecks, techniques like lock stripping - using separate locks per warehouse or SKU batch and very short lock timeouts can be applied. While this may slightly impact throughput, it guarantees accuracy and correctness in inventory management.

Jonathan Gomes(9900)

# Assignment 2

Case:- An ecommerce platform experiences latency during flash sales.

Q1: How can distributed caching and load balancing improve performance?

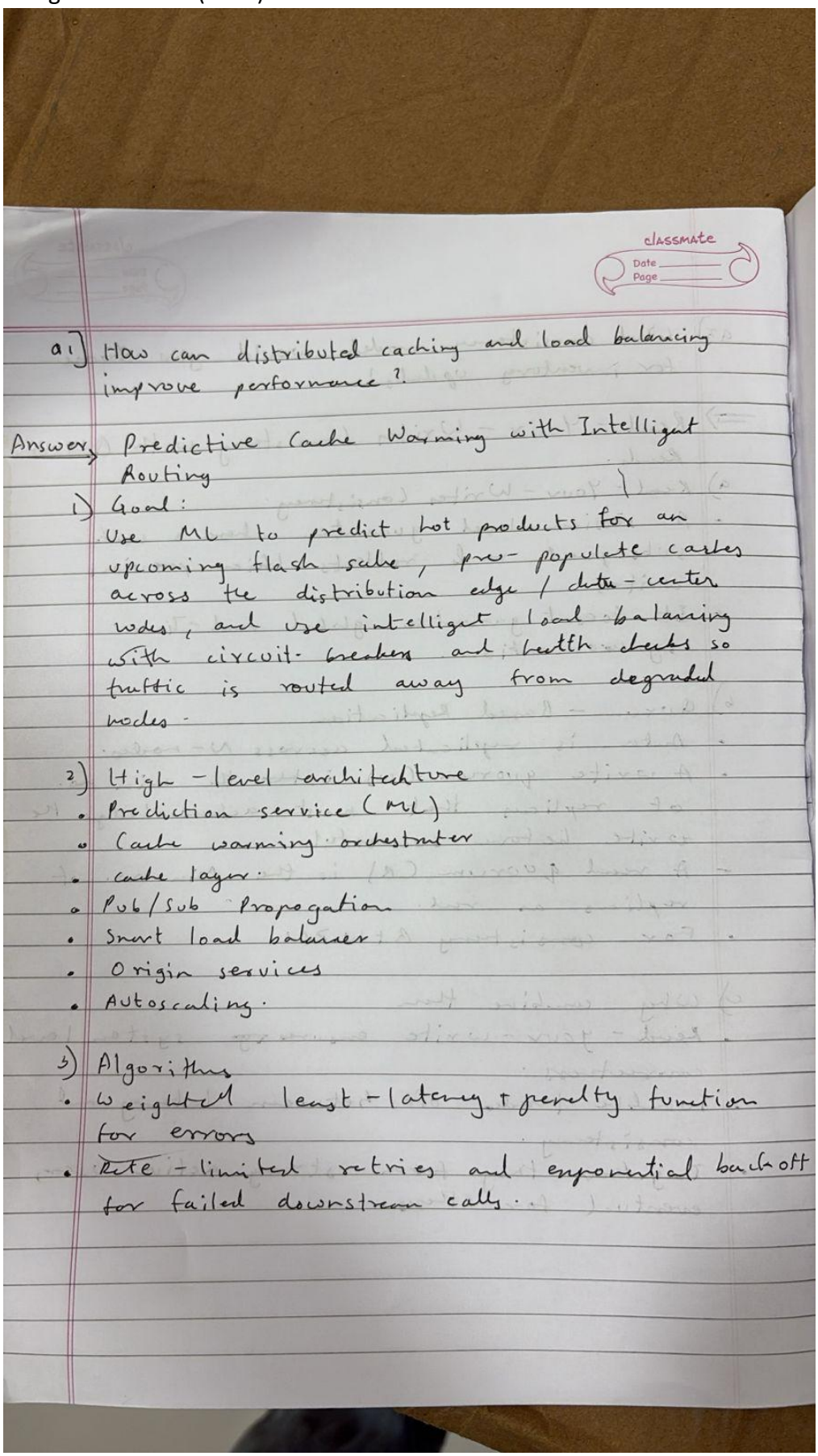
→ Read-Write Split with Cache-Aside Pattern:

Seperate read and write operations using load balancers that route to different server pools. Read-heavy operations (product browsing, search) hit cached replicas, while writes (purchase, cart updates) go to primary databases, updating cache on miss. This reduces database load by 90% + with cache hits returning data in sub-millisecond latency vs database queries taking 10-100 x longer. The architecture enables horizontal scaling by adding read-replica servers and cache nodes as traffic increases, handling 10x normal traffic without overwhelming backend systems. Configure cache TTL based on data volatality product catalogs can have longer TTLs (hours) while inventory counts need shorter TTLs to prevent serving stale data.

Q2. What consistency model would you choose for inventory updates?

→ Implement causal consistency where inventory updates preserve happened-before relationships. using version vectors that track write counts per data item. Each inventory record maintains a vector storing version numbers at each replica node - when an update occurs, the corresponding element increments and propagates with update message. Users see their own updates immediately and causally related updates within the same user session appear in correct order, preventing scenarios where customers see outdated stock after completing purchases.

Mrugank Worlikar(9938)



a1] How can distributed caching and load balancing improve performance?

Answer → Predictive Cache Warming with Intelligent Routing

1) Goal:
Use ML to predict hot products for an upcoming flash sale, pre-populate caches across the distribution edge / data-center nodes, and use intelligent load balancing with circuit-breakers and health checks so traffic is routed away from degraded nodes.

2) High-level architecture
- Prediction service (ML)
- Cache warming orchestrator
- cache layer.
- Pub/Sub Propagation
- Smart load balancer
- Origin services
- Autoscaling.

3) Algorithms
- weighted least-latency + penalty function for errors
- Rate-limited retries and exponential backoff for failed downstream calls.

q2) What consistency model would you choose for inventory updates?

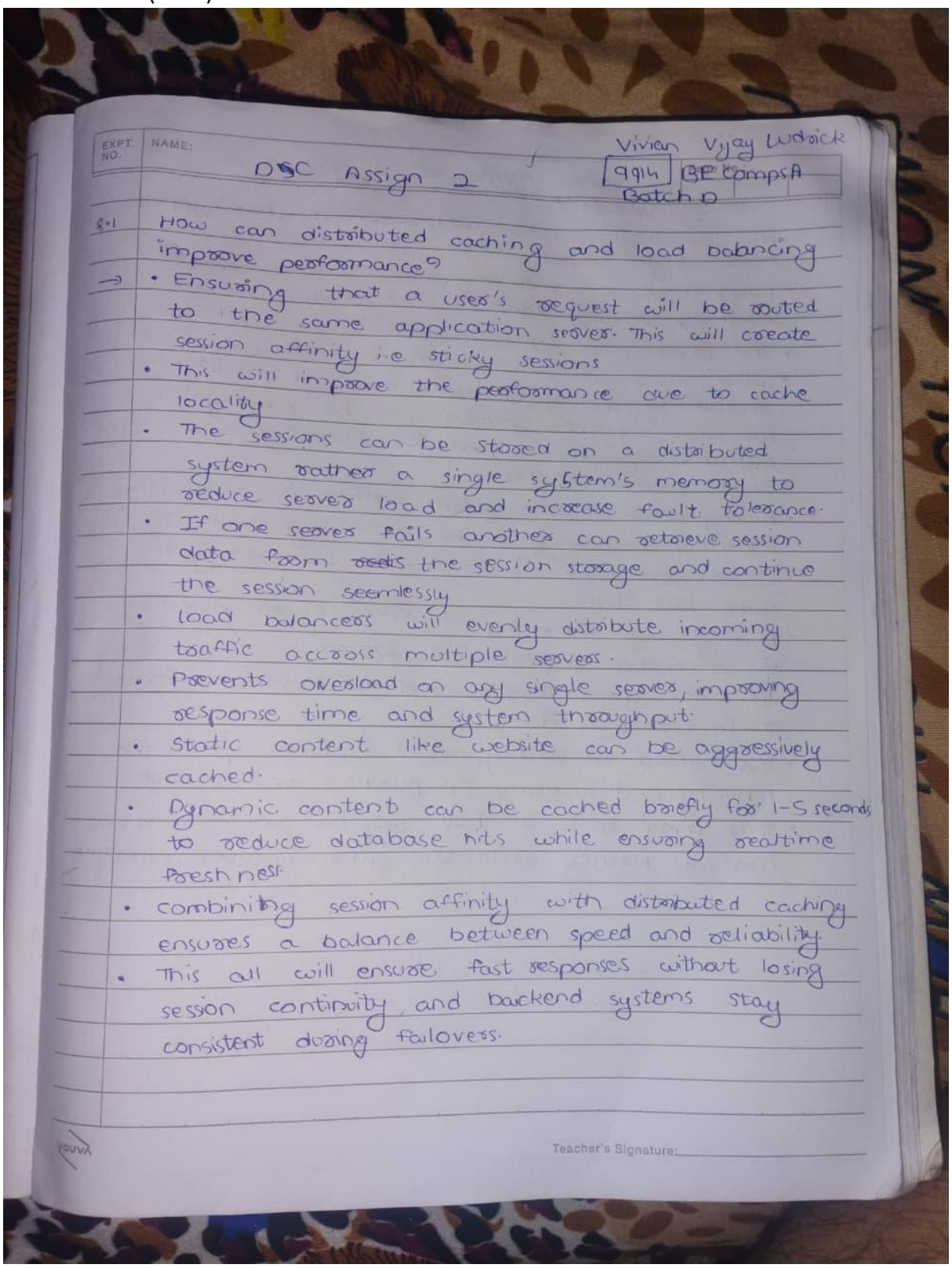
⇒ Read - Your - Writes Consistency with Quorum Reads.

a) Read - Your - Writes Consistency:
- A session - level guarantee where a user's subsequent reads reflects thier previous writes
- It doesn't garuntee global real - time synronization.

b) Quorum - Based Replication
- Data is replicated accross N - nodes.
- A write quorum (W) is the min no. of replicas that must acknowledge the write before successful.
- A read quorum (R) is the Min no. of replicas as read.
- For consistency R + W > N

c) Why combine them
- Read - your - write ensures system level correctness.
- Tunable trade - offs between latency and consistency.
- Together they form strong active user, eventual for others.

Vivian Ludrick(9914)



EXPT. NO. NAME: Vivian Vijay Ludrick

9914 | BE Comps A | Batch D

DSC Assign 2

Q.1 How can distributed caching and load balancing improve performance?

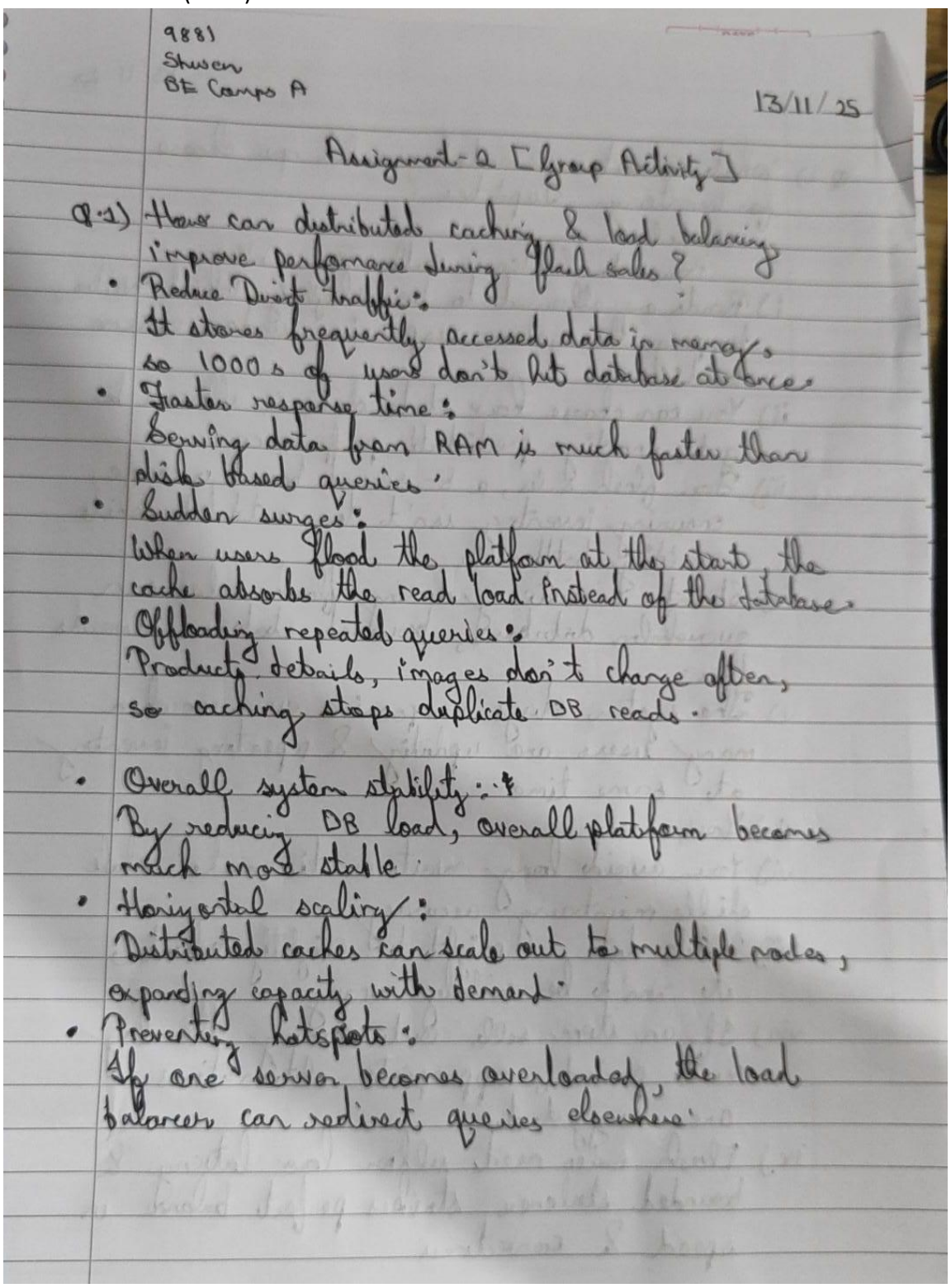
→
- Ensuring that a user's request will be routed to the same application server. This will create session affinity i.e sticky sessions
- This will improve the performance due to cache locality
- The sessions can be stored on a distributed system rather a single system's memory to reduce server load and increase fault tolerance.
- If one server fails another can retrieve session data from ~~redis~~ the session storage and continue the session seemlessly
- load balancers will evenly distribute incoming traffic accross multiple servers.
- Prevents overload on any single server, improving response time and system throughput.
- Static content like website can be aggressively cached.
- Dynamic content can be cached briefly for 1-5 seconds to reduce database hits while ensuring realtime freshness
- combining session affinity with distributed caching ensures a balance between speed and reliability.
- This all will ensure fast responses without losing session continuity and backend systems stay consistent during failovers.

Teacher's Signature:

Q 2. what consistency model would you choose for inventory updates?

→ • All inventory updates (add, reserve, purchase) appear in a single global order.
- Every user will see operations in the same sequence, avoiding conflicts like double selling.
- The reservation can be divided into 2 phases.
- When user add an item to a cart or check availability, the system uses eventual consistency for quick response. Updates will be propagated asynchronously. This is good for browsing and casting large number of items.
- During checkout, the system verifies inventory using sequential consistency before confirming payment.
- This guarantees that no two users can buy the same last item.
- During high demand peak times, excess purchase or reservation requests are queued.
- This will ensure fairness and avoids race conditions on limited stock items.
- This hybrid consistency model combines speed for browsing and accuracy for purchase confirmation
- This gives best of both worlds where the user has low latency while the system ensures correctness.

Shwen Coutinho(9881)



9881
Shwen
BE Comps A

13/11/25

Assignment-2 [Group Activity]

Q-1) How can distributed caching & load balancing improve performance during flash sales?

- Reduce Direct traffic:
  It stores frequently accessed data in memory, so 1000's of users don't hit database at once.
- Faster response time:
  Serving data from RAM is much faster than disk based queries.
- Sudden surges:
  When users flood the platform at the start, the cache absorbs the read load instead of the database.
- Offloading repeated queries:
  Products details, images don't change often, so caching stops duplicate DB reads.
- Overall system stability:
  By reducing DB load, overall platform becomes much more stable.
- Horizontal scaling:
  Distributed caches can scale out to multiple nodes, expanding capacity with demand.
- Preventing hotspots:
  If one server becomes overloaded, the load balancer can redirect queries elsewhere.

13/11/25

Q.2) What consistency model would you choose for inventory updates?

Model:

Bounded Staleness with Time Windows.

i) Reads are allowed to be slightly behind writes, keeping the system responsive under extreme load.

ii) You can choose how stale data is allowed to get, enabling predictable performance

iii) For flash sales, a 5-second bound is made, ensuring inventory isn't too outdated while avoiding bottlenecks.

iv) Strict consistency during flash sales can overwhelm databases, bounded staleness avoids this.

v) The system stays responsive even when many users are updating & repeating inventory at same time.

~~vi) You avoid~~

vi) You avoid long waits or timeouts while still maintaing accuracy.

vii) While data might be slightly behind, it won't be wildly wrong.

viii) If an item sells out during the lag wait period, appropriate alternatives are suggested.

ix) Flash sales need ultra low latency & bounded staleness strikes perfect balance in speed & correctness